Windows execution guide

Design and Implementation of a 2×2 Mesh Network-on-Chip Router Using VHDL on FPGA

Anjor Giri · Nidhi Kanoujia · Deven Gaikwad
Faculty and mentor: Dr. Madan Mali
Vivado 2026.1 / XSim · xc7s15cpga196-2 · target runs pending

Reference environment: AMD Vivado 2026.1 with XSim; VHDL-2008; exact part xc7s15cpga196-2. The software package includes actual GHDL evidence, but local Vivado simulation/synthesis/STA are still pending. This guide records each gate separately. A board is not needed to start simulation.

1. Extract and separate source from generated project

Goal: keep editable sources safe. In File Explorer, right-click the delivered ZIP, choose Extract All, and place its vhdl-2x2-mesh-noc folder under E:\Vivado_Projects\NoC_2x2_Source. Avoid OneDrive-controlled folders and unusual path characters. Generated Vivado files belong under E:\Vivado_Projects\NoC_2x2. Do not overwrite the source folder with generated project files.

Expected: rtl, tb, constraints, vivado, scripts, frontend, docs, report, paper, presentation and evidence directories exist. Open README.md and PROGRESS.json. Save a copy of the package outside the working folder. If a referenced file is absent, extract the whole ZIP rather than copying one VHDL file.

2. Confirm Vivado and the exact part

Goal: use the requested tool/device. Launch Vivado 2026.1 from Start. Open Help > About and record the full build. In the Tcl Console run:

```
version -short
get_parts xc7s15cpga196-2
```

Expected: exactly the requested part is returned. Save the console capture and tool version under evidence/vivado. If the part list is empty, modify the installed device support to include Spartan-7; do not silently choose a larger chip or a Zynq part. XC7S15CPGA1962 is only a colloquial spelling of the normalized tool string.

3. Create the project in the GUI

Goal: establish the hierarchy. On the welcome page click Create Project > Next. Name it NoC_2x2 and select E:\Vivado_Projects as location, with a project subdirectory. Choose RTL Project. Add all design sources in rtl/compile_order.txt order; enable copying sources only if you deliberately want a separate snapshot. Add clock_only.xdc as the constraint file. Do not add board_pins_TEMPLATE.xdc.

In Default Part choose Parts, family Spartan-7, and the exact xc7s15cpga196-2 device/package/speed grade. Finish. Expected: Sources shows the design files and Project Summary shows the exact part. Save the .xpr in the generated project directory and capture the part selection. If Vivado guesses a wrong hierarchy, set tops explicitly in step 5.

4. Set VHDL-2008 and add testbenches

Goal: compile the intended language. In Sources select each .vhd, inspect Source File Properties and set File Type to VHDL 2008. Right-click > Add Sources > Add or Create Simulation Sources; add every tb/*.vhd. Mark testbenches as simulation-only. The package must compile before modules using its types/functions.

Expected: Simulation Sources contains tb_noc_2x2 and module tests; design hierarchy contains no TextIO/finish testbench. Save a compile-order screenshot. If process(all), std.env or array ports fail at analysis, first check VHDL-2008/file type and package order. Do not replace numeric_std with legacy arithmetic packages.

5. Set different design and simulation tops

Goal: avoid synthesizing the testbench. Under Design Sources right-click noc_2x2_core > Set as Top. Under Simulation Sources right-click tb_noc_2x2 > Set as Top. Default simulation generics are DEPTH=4, SCENARIO=0 and TRACE_LEVEL=2.

Equivalent console commands:

```
set_property top noc_2x2_core [get_filesets sources_1]
set_property top tb_noc_2x2 [get_filesets sim_1]
set_property generic {DEPTH=4 SCENARIO=0 TRACE_LEVEL=2} [get_filesets sim_1]
update_compile_order -fileset sources_1
update_compile_order -fileset sim_1
```

Expected: the diamond/top indicators differ for design and simulation. Save this screenshot. If synthesis mentions std.textio or std.env.finish, the wrong top/source set is being synthesized.

6. Use the reproducible Tcl setup instead of repeated GUI entry

Goal: reproduce project setup without losing sources. Open a Vivado Tcl shell or an ordinary PowerShell where vivado is on PATH. Change to the extracted repository root and run:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\scripts\run_vivado.ps1 -Task Create -ProjectDir 'E:\Vivado_Projects\NoC_2x2'
```

The first command affects only this PowerShell session. If Vivado is not on PATH, open the installed Vivado Command Prompt or pass -VivadoCommand with the actual full path to vivado.bat. Do not assume another machine's installation directory.

Equivalent batch invocation:

```
$env:NOC_PROJECT_DIR='E:\Vivado_Projects\NoC_2x2'
vivado -mode batch -source .\vivado\create_project.tcl
```

Expected: part availability and project path print; existing matching .xpr is opened without deleting it. Save evidence/vivado/Create.log. This is a setup success, not a simulation PASS. If an existing project targets another part, inspect it separately; the script stops rather than changing it silently.

7. Run the directed depth-four XSim test

Goal: verify the same protocol in the required simulator. In Flow Navigator click Run Simulation > Run Behavioral Simulation. After loading, click Run All or use run all in the simulation Tcl Console. The testbench ends itself; arbitrary stop-time screenshots are not completion evidence.

For a trace saved directly in the evidence directory, use:

```
.\scripts\run_vivado.ps1 -Task Simulate
```

Expected: console contains PASS tb_noc_2x2 and no severity failure. With this exact RTL/configuration the prior GHDL result was 106 accepted, 74 delivered, 32 reset-discarded and zero unexplained outstanding. A different XSim output must be investigated, not copied from GHDL. The script writes an actual trace under evidence/vivado/xsim/directed-d4/trace.csv. Save the real console and simulator log before rerunning.

If the generic TRACE_FILE cannot open, check directory existence and the quoted absolute string in run_sim.tcl. Default GUI TRACE_FILE=trace.csv writes into XSim's working directory, typically the generated .sim/sim_1/behav/xsim directory; locate the actual file before importing it. Do not fabricate a CSV when a run fails.

8. Inspect actual waveforms and stalls

Goal: connect assertions to signal behavior. In the simulation hierarchy expand tb_noc_2x2 > dut. Add clk, rst, tv/tr injection valid/ready, rv/rr ejection valid/ready, tx/rx packet arrays, count, req and grant. The supplied vivado/waves.tcl adds named signals when available. Set packet data radix to hexadecimal and zoom around a known accepted injection or stall.

Expected: on a stalled output, the held one-hot grant and complete data stay fixed until valid and ready are both high at a rising edge. A full FIFO refuses a push even if it pops on that edge; occupancy then permits retry. Newly stored input data cannot forward on the same edge. Save File > Save Waveform Configuration and a genuine waveform capture after the run. Keep clock/rst and relevant handshake visible; include run/configuration in the filename. The supplied CSV-derived timing table is explanatory data, not an XSim screenshot.

If a waveform contains U/X after reset, preserve the failure and inspect reset, connection drivers and initial deltas. Do not hide warnings or treat a static in-band marker as a handshake.

9. Run all fourteen XSim verification configurations

Goal: validate FIFO, route, arbitration, every corner and the mesh independently.

```
.\scripts\run_vivado.ps1 -Task Verify
```

Expected: each actual log contains its top-specific PASS. The script checks logs and stops on a missing PASS/failure. Tests are FIFO depths 1/3/4/8, route, arbiter, routers 0/1/2/3, demo, mesh depths 4/2/8. Logs copy under evidence/vivado/xsim. If a Vivado build writes xsim.log at another location, inspect that observed location and adjust the script's log lookup; do not bypass the PASS check.

Save all logs and actual version/part before marking XSim verified in PROGRESS.json. A syntax-checked Tcl file is not proof that XSim ran. Fix the first compiler error in the source/dependency chain, then relaunch; keep a known working project copy instead of deleting all evidence.

10. Elaborate RTL and inspect hardware hierarchy

Goal: see the design that will synthesize. Return to the main Vivado project; design top must still be noc_2x2_core. In Flow Navigator click Open Elaborated Design. Inspect four router tiles and the active corner FIFOs/arbiters. Do not expect real CPU/AI cores: endpoints are traffic models only in the demo top.

Expected: no undriven/multiply driven control, unintended black box or testbench hierarchy. Save the elaborated schematic and all warnings. If unused boundary logic remains in a displayed logical five-port view, inspect the generate-time masks rather than assuming five physical neighbors. Synthesis later decides exact physical inference.

11. Synthesize the logical core out of context

Goal: measure exact-device fabric cost without pretending wide logical ports fit the board.

```
.\scripts\run_vivado.ps1 -Task Synthesize
```

run_synth.tcl reads VHDL-2008, synthesizes noc_2x2_core for xc7s15cpga196-2 in out_of_context mode, applies clock_only.xdc and saves actual reports under evidence/vivado/synthesis_ooc. The checkpoint is generated under build/checkpoints.

Expected: actual utilization, hierarchical utilization, check_timing, timing summary, critical-path and DRC reports. Open them and inspect inferred storage, LUT/FF/BRAM, warnings, latch reports and unused logic. Save the synthesis log and selected part. A GHDL inferred RAM note does not mean AMD Block RAM is inferred.

GUI Run Synthesis can inspect normal project synthesis, but its logical external I/O treatment is different from this OOC experiment. The core's four 32-bit interfaces plus debug arrays are not a feasible package pin assignment. An I/O fit warning is not fixed by inventing pins or changing the target. Record which top/mode/depth each resource figure measures.

12. Understand the clock and unconstrained paths

Goal: make timing claims with the right scope. clock_only.xdc defines create_clock -period 10.000 on clk, a 100 MHz design target. This is not a measured oscillator or fMAX. No fabricated input/output delays, pin locations or I/O standards are supplied.

Expected: check_timing reports missing external constraints where applicable. Internal register-to-register paths can be analyzed against the target clock, while top-level packet/reset/start/status timing is not a complete board model. Save clock definitions and all unconstrained-path warnings. Do not use a blanket false path to make a red report green. External timing constraints need actual board/interface information and justified synchronization.

13. Route the small demo fabric and read STA

Goal: examine internal routed delay without a physical-board claim.

```
.\scripts\run_vivado.ps1 -Task RouteDemo
```

This synthesizes noc_fpga_demo_top in OOC mode, runs opt_design, place_design and route_design, and writes reports under evidence/vivado/route_demo_ooc. It includes traffic models, so its utilization is different from core-only synthesis. It does not insert a verified board pinout, validate I/O timing or generate a bitstream.

Expected: actual completion or explicit placer/router failure. Preserve either result. In Timing Summary inspect WNS, TNS, failing endpoints, hold timing, path groups and unconstrained paths. WNS<0 means at least one constrained setup path misses the target; TNS aggregates negative setup slack. WNS≥0 is meaningful only for covered paths and does not prove board operation or maximum frequency. Open the worst path and record launch/capture registers, logic levels, routing delay and slack. Do not assert “timing closure” while ignoring unconstrained interfaces or hold failures.

VHDL timing simulation is not supported by Vivado 2026.1. Use behavioral/valid post-synthesis functional XSim as functional evidence and post-route static timing as timing evidence. No SDF/VHDL timing waveform is claimed.

14. Import actual Vivado summaries into the dashboard

Goal: use real report values. After a successful local run and manual warning/constraint review:

```
node .\scripts\import_vivado_reports.mjs .\evidence\vivado\synthesis_ooc
# Or the route_demo_ooc directory after that separate run.
```

Expected: summary.json with exact part/tool/stage, report hashes and recognized numbers. Unrecognized table headings or absent slack remain null. Import this file through the dashboard's Import trace or metrics button. The browser labels it user-supplied and does not authenticate the tool; raw reports remain authoritative. Do not fill null values with invented estimates.

15. Install Node and open the local dashboard

Goal: demonstrate without Python or a connected FPGA. Install the official Node.js 24 LTS version compatible with the pinned 24.19.0 baseline. Install VS Code if useful. Open the project's frontend folder in VS Code and choose Terminal > New Terminal.

```
node --version
npm ci
npm test
npm run dev
```

Expected: parser tests pass and Vite prints http://127.0.0.1:5173. Open that address. Load GHDL sample to inspect the actual supplied depth-four trace, or import your real XSim CSV. Use Replay to move exact edges, Router inspector to read recorded state, Packet inspector to decode the complete word, Verification for evidence, and Performance to load the 93-run summary. All files stay in browser memory.

If npm is unavailable, restart the terminal after installation and check PATH. If PowerShell rejects npm.ps1, invoke npm.cmd or use a process-only policy. If the port is busy, close the previous local Vite process or choose a documented free port; do not open a random public service. If an import fails, inspect the exact schema/header and run ID rather than disabling validation.

16. Test the production application

Goal: ensure the static build works.

```
npm run build
npm run preview
```

Expected: dist contains index.html, assets and bundled data. Open the preview HTTP URL and check all nine views. Opening file://index.html directly is unsupported because browser module/fetch rules differ. The relative base supports later static hosting, but GitHub Pages/public visibility are separate choices and were not enabled. A frontend build PASS certifies the app check, not FPGA timing.

17. Preserve and share the private Git repository

Goal: preserve source history and use secure authentication. Follow docs/github_workflow.md. Install Git/GitHub CLI and complete gh auth login through the browser. Confirm gh auth status and username. Clone repository.bundle into a fresh folder; remove the bundle-path origin with git remote remove origin before private repository creation. Alternatively, initialize the extracted source once. Review .gitignore, git status and staged diff before push.

```
gh auth status
gh repo create vhdl-2x2-mesh-noc --private --source=. --remote=origin --push

git remote -v
git rev-parse HEAD
gh repo view --json url,visibility,defaultBranchRef
gh run list --limit 5
```

Use the creation command only after confirming an intended matching repository does not already exist. Expected: observe the real URL, PRIVATE visibility, main branch and pushed commit. Do not invent a URL

before success, force-push unrelated history or expose a token. Hosted Actions checks have not run here. Public visibility, Pages, collaborators and paper publication require a separate choice.

18. Board-specific implementation gate

Goal: eventually demonstrate real FPGA behavior. First obtain the exact manufacturer board model/manual, oscillator frequency and package pin map for this device, bank voltages/I/O standards, reset/start circuitry, accessible status outputs and actual USB/JTAG programmer. Replace the deliberate placeholders in board_pins_TEMPLATE.xdc only with verified information; create a real reviewed board-specific XDC and keep the template untouched.

Implement proper conditioning for asynchronous reset/start, map only accessible outputs, justify external delays/clock constraints, run synthesis/place/route/DRC/STA on that board top, then Generate Bitstream and Open Hardware Manager > Open Target > Auto Connect only when the real board/programmer is identified. Verify the device, select the resulting .bit and Program Device. Save actual programming logs and physical status observations.

These later clicks describe the gate, not completed work. No usable bitstream exists in this package. Device identification alone does not identify a board, oscillator, USB driver or pin allocation. Missing board information does not block the software replay or OOC core analysis.

Ten-minute faculty demonstration

1. Minute 0–1: state title, four traffic models, exact part and verified-versus-pending tool status.
2. Minute 1–2: show canonical topology and both two-hop XY examples.
3. Minute 2–3: decode marker/type/X/Y/ID/priority/payload and explain accepted ready/valid.
4. Minute 3–5: show actual trace replay or the locally rerun XSim waveform; demonstrate held data/grant through stall and explain the bubble.
5. Minute 5–6: show fourteen PASS logs, all-pair/all-link assertions and explicit reset accounting.
6. Minute 6–8: compare actual all-to-all and saturated FIFO-depth results; explain cohort/window metrics and conditional generation gate.
7. Minute 8–9: show real local Vivado reports if executed, otherwise the pending state and exact next scripts. Do not substitute GHDL for FPGA timing.
8. Minute 9–10: explain limitations, source history and next board gate; answer using the one-page viva sheet.

Troubleshooting reference

Symptom	Likely cause / safe response
Part unavailable	Install requested Spartan-7 device support; retain exact part
VHDL syntax/type failure	Check package-first order, VHDL-2008 and first error
TextIO/finish during synthesis	Testbench is wrongly in design source set/top
Trace missing	Check quoted TRACE_FILE and actual XSim working directory
Assertion failure / missing PASS	Preserve log; investigate handshake, route, order and reset; do not mark passed
FIFO appears full one extra edge	Intended registered-capacity boundary, not combinational look-ahead
High I/O count	Wide logical core/debug ports are not a board interface; use OOC analysis
Negative WNS / unknown timing	Inspect critical path and constraint coverage; do not invent false paths
DRC NSTD/UCIO / unknown board	Supply verified board constraints before bitstream work
npm.ps1 blocked	npm.cmd or process-only policy; no machine-wide relaxation
CSV rejected	Verify exact 16-column schema, packet fields, one run and nondecreasing cycles
GitHub auth unavailable	Use secure browser sign-in on your machine; never paste a token into chat